

Sa Optimal Inplace

最优原地后缀排序算法

本章介绍线性时间复杂度的后缀排序的就地算法¹（Optimal In-Place Suffix Sorting）。

▼ Warning

本章 只建议 在 非常非常熟悉 SA-IS²³的前提下阅读。

全局设定

目标字符串，后缀数组，串的序号从 0 开始，结尾字符是警戒哨，不妨设为 0。

在整形字母表上的后缀排序

事实上这一部分可以看成是原地版本的 SA-IS 算法。

因为是原文中细节相对最清楚，实现也较为简单的算法，也是了解后续算法的基础，是本文介绍的重点。

原地化的原理是用重命名的 代替 S、L 桶，用额外 的操作代替类型桶。

重命名目标串 Pat

简单来说，我们会在不改变后缀大小的相对顺序的前提下，重命名，用重命名后的 来取代原来 S、L 桶，来指明桶头或者桶尾。

重命名的方法是 将 中的 S 型字符替换为所在桶的桶尾索引，L 型字符替换为所在桶的桶头索引。

如下图所示：

重命名后的 （之后直接将重命名后的 称做）：

由于桶内的字符，L 型字符后缀小，作为桶头；而 S 型字符后缀大，作为桶尾，因此保持了后缀大小的相对顺序。

描述一下重命名的具体步骤：

1. 和 SA-IS 一样，对 中每个字符计数，计算其前缀和（计数排序），来构建 S/L 桶，只不过这里用 盛放这个前缀和；
2. 从尾到头，扫描 的每个字符，这样只需记录上一个字符的类型，就可以动态地判断每个字符的类型，然后依据前缀和将其重命名。

对 LMS 字符排序

这里重点是使用了一个内部计数器的技巧。

初始化

初始的时候将 每一项设为 E（EMPTY）。

从尾到头扫描，如果发现是 LMS 字符，，那么就设置 的标记：

如果 是 E，就将其设为 U (UNIQUE) ；

如果 是 U，就将其设为 M (MULTIPLE) ；

其他情况，不做处理。

结果如下图所示：

把 LMS 字符的索引放入 SA

从尾到头扫描，对于 LMS 字符，根据 的符号进行分类讨论：

U：直接让

M：意味着桶中有至少两个 LMS 字符。

1. 如果桶中有至少三个 LMS 字符：就把桶中倒数第二个位置作为临时计数器，标志桶中已填充的 LMS 字符数（桶中倒数第一位就是标志 M）将新的 LMS 字符从倒数第三个位置开始插入，让临时计数器自增 1。如果发现桶已经满了，就把桶中从桶头到倒数第三个的所有元素向右平移 2 个位置，然后把新元素插入到桶中第二个位置（桶中第一个位置填为 E）
2. 如果桶中有且只有 2 个 LMS 字符，显然不需要计数器，直接从右到左顺序插入即可。

正常的值：

- 1 根据我们之前的讨论，此时不管桶中有两个还是两个以上的 LMS 字符，这都意味着 $\text{texttt}\{i\}$ 是桶中最后一个待插入的 LMS 字符的位置，
- 2
- 3 只需要从桶头开始向左扫描，找到第一个标记为 E 的位置，将其设为 $\text{texttt}\{i\}$ 。

最后要从尾到头扫描一遍，清除可能残余的特殊符号 M（桶中未被填满，所以 M 和计数器未被覆盖）。

方法是将桶中 LMS 字符如上述步骤一样向右平移 2 位，将左边空出来的位置填为 E。

如下图所示：

这个阶段，由于每个桶只需要被移动和扫描一次，所以时间复杂度是 。

诱导排序 LMS 子串

诱导排序 LMS 前缀

将 LMS 前缀进行诱导排序，同 SA-IS 一样，这部分同后面对后缀的诱导排序完全一样（使用同一个函数），因此这里直接跳过。

这里直接给出排序结果：

将已排序的 LMS 子串放到 SA 尾部

构建规模缩减的子目标串 Pat1

从左到右扫描 尾部的 LMS 子串，确定其大小关系「重命名」，将 重命名的值存储在 。

因为 LMS 字符并不相邻，所以不会有冲突，这样做是将重命名后的值按照所代表的子串在 中的原顺序放置：

然后扫描，收集这些重命名的值到 头部：

通过递归解决 Pat1，完成对 LMS 后缀的排序

同 SA-IS 一样，递归解决 头部的规模缩减的 的后缀排序，结果存到 尾部：

将 尾部的 挪到 头部，重新从尾到头扫描，将其中 LMS 字符按照在 中的顺序放到 尾部：

依照 尾部的「对照表」，将 头部的 还原为 中对应的 LMS 后缀的索引位置：

将 头部的排好序的 LMS 后缀按顺序放入到对应的桶中（从尾部开始放）：

对 Pat1 中所有的后缀进行诱导排序

这一部分就是利用前面用过的内部计数器技巧，进行原地版的诱导排序。

假如我们已经排好序的 LMS 后缀（在桶尾），来诱导 L 型后缀⁴：

如同排序 LMS 字符一样，先对 L 型字符用特殊符号计数：

从左到右扫描 SA，同对 LMS 字符排序一样，复杂一点的是判断 的类型，需要分类讨论（详情参考代码）：

区别于 SA-IS 的是，对一个类型字符诱导排序后，需要清理 LMS 字符以免对后面的原地诱导排序：

至于从 L 后缀诱导 S 后缀与从 LMS 后缀诱导 L 后缀完全对称，这里就不做多余介绍。

到这儿为止，诱导排序就完成了。

实现

时间性能上和 SA-IS 没有显著差别，空间占用变为不到原来的 （代码量多 1 倍），算是不愧为原文 Optimal In-Place Suffix Sorting¹的标题。

► 参考代码

在只读的整形字母表上的后缀排序

使用复杂方法解决复杂问题，通过分治，解决空间紧张的问题。

算法实现的难点在于在上构建 BitMaps⁵，来替代本来由重命名后的 T 所指示的指示桶尾/桶头的位置。

这里的 BitMaps 指得是使用比特向量（bit vector）表示的有序字典（multiset），是一种紧凑型结构（compact data structure）。

有兴趣了解的暂时只能阅读原文以及本文引用的 BitMaps 的有关论文自行了解。

在只读的一般字母表上的后缀排序

前置知识是归并排序和堆排序。

由于笔者对于其中确定字符类型的方法的时间复杂度有疑问，这里也不再介绍，建议阅读原文自行了解。

注解

-
1. Li, Zhize; Li, Jian; Huo, Hongwei (2016). *Optimal In-Place Suffix Sorting*. Proceedings of the 25th International Symposium on String Processing and Information Retrieval (SPIRE). Lecture Notes in Computer Science. 11147. Springer. pp. 268–284. arXiv:1610.08305. doi:10.1007/978-3-030-00479-8_22. ISBN:978-3-030-00479-1. [↩↩](#)
 2. Ge Nong, Sen Zhang, and Wai Hong Chan. Linear suffix array construction by almost pure induced-sorting. In Data Compression Conference (DCC), pages 193–202. IEEE, 2009. [↩](#)
 3. 推荐阅读 [博文](#) 和它的 [issue 列表](#) [↩](#)
 4. 如果是 LML 后缀，就先诱导 S 型后缀，唯一区别是计算 LML 后缀时需要将警戒哨也算进去。 [↩](#)
 5. Gonzalo Navarro and Eliana Provedel. Fast, small, simple rank/select on bitmaps. In Proc. 1st International Symposium on Experimental Algorithms (SEA), pages 295–306, 2012. [↩](#)
-

本页面最近更新：2023/5/6 19:22:00, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者：[billchenchina](#), [CCXXI](#), [Enter-tainer](#), [ksyx](#), [minghu6](#), [ouuan](#), [Tiphereth-A](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用